

Day 53: Residual Connections

MD Mutasim Billah | 52-Week Data Science → ML/AI Roadmap | Week 8: Classic CNN Architectures

0. Where Today Fits

Day 52 ended on a cliffhanger: VGGStyleConvNet (4 conv+ReLU layers) failed to train at all at $lr=0.15$ because ~99.9% of a late conv layer's units went permanently dead under ReLU. A learning rate tuned for a shallower network was too aggressive for a deeper one. Today builds the fix He et al. (2015, the ResNet paper) actually proposed: give each block an identity shortcut around its convs.

Start here (no prior knowledge assumed):

A normal conv block computes $out = ReLU(F(x))$, where F is two stacked convolutions. A residual block instead computes $out = ReLU(F(x) + x)$ -- it adds the block's own INPUT back to its output before the final activation. Since $d(F(x)+x)/dx = dF/dx + 1$, gradient flowing backward always has an unattenuated "+1" path straight through, on top of whatever the conv path contributes.

Watch: [ResNet Explained Simply \(Skip Connections Guide\)](#)

Watch: [Vanishing & Exploding Gradient explained | A problem resulting from backpropagation](#)

1. The Residual Block, Exactly

SYSTEM DIAGRAM

Plain block: $x \rightarrow \text{conv1} \rightarrow \text{ReLU} \rightarrow \text{conv2} \rightarrow \text{ReLU} \rightarrow \text{out}$

Residual block: $x \rightarrow \text{conv1} \rightarrow \text{ReLU} \rightarrow \text{conv2} \rightarrow (+x) \rightarrow \text{ReLU} \rightarrow \text{out}$

Note where the addition happens: AFTER conv2, BEFORE the final ReLU -- matching He et al.'s original design exactly. Both convs use padding=1 (new today) so a 3x3 conv preserves spatial size, which is required for x and $F(x)$ to have matching shapes to add together.

MATH

Backward pass for one residual block, given upstream gradient $dout$:

$dsum = dout * \text{relu}'(z2 + x)$ (undo the outer ReLU)

$dz2 = dsum$ (sum's gradient flows unchanged into $z2$)

$dx_identity = dsum$ (AND unchanged straight into x -- the shortcut path)

... backward through conv2, relu, conv1 along the normal path gives dx_from_conv1 ...

$dx = dx_from_conv1 + dx_identity$ (the two paths' gradients simply ADD)

2. Two Networks, Identical Parameter Counts

PlainDeepConvNet and ResNetStyleConvNet share the exact same architecture -- a stem conv ($3 \rightarrow C$) plus 3 blocks of 2 convs each ($C \rightarrow C \rightarrow C$), 2×2 maxpool after every block -- 7 conv+ReLU layers total, nearly double Day 52's deepest network. The ONLY difference is whether each block adds its input back before the final ReLU. A residual shortcut adds zero parameters, so both networks have identically 7,012 trainable parameters.

3. Gradient Check -- Does the Shortcut's Backward Math Work?

Splitting the upstream gradient into a conv-path term and an identity-path term, then re-summing them into dx, is new math Days 39-52 never needed. It was checked against numerical (central-difference) gradients across every weight tensor before either network was trusted to train.

ACTUAL OUTPUT

```
max relative error across all checked entries: 1.02e-07 (PASS -- residual-block backward chain is correct)
```

4. Gradient Flow by Depth -- the Direct Vanishing-Gradient Diagnostic

Day 52 diagnosed dying ReLUs AFTER training -- a downstream symptom. Today measures the cause directly: the gradient norm reaching each conv layer's weights on a single forward+backward pass at initialization, deepest layer first. Both networks start from IDENTICAL initial weights (same seed), so any difference is due to the shortcut connections alone.

ACTUAL OUTPUT

```
layer (deepest first) plain ||dw|| resnet ||dw||
block2.W2 15.053117 7.472742
block2.W1 20.450470 9.903194
block1.W2 20.776196 14.258992
block1.W1 10.851486 11.091479
block0.W2 3.888616 5.766880
block0.W1 4.234454 4.991802
stem 1.744730 3.104412

stem-to-deepest-layer gradient ratio -- plain: 0.1159, resnet: 0.4154
```

PlainDeepConvNet's earliest layer (the stem) retains only 11.6% of the deepest block's gradient magnitude by the time gradient reaches it; ResNetStyleConvNet retains 41.5% -- a real, measured effect of the shortcut connections, confirmed on two networks that started from bit-identical weights.

5. First Attempt -- Same lr=0.15 That Broke Day 52's Network

ACTUAL OUTPUT

```
network params train_acc test_acc
PlainDeepConvNet 7012 0.2500 0.2500
ResNetStyleConvNet 7012 0.2500 0.2500

dead fraction (block's second conv, stuck <= 0 after training):
block0.Zc2 plain: 1.000 resnet: 1.000
block1.Zc2 plain: 0.875 resnet: 1.000
block2.Zc2 plain: 1.000 resnet: 1.000
```

Both networks fail completely -- chance-level accuracy for both. Residual connections did NOT rescue this learning rate on their own. More gradient reaching a layer (the diagnostic above) is not the same as a smaller update step once that gradient arrives: lr=0.15 is still large enough to push units permanently negative in both networks. A shortcut connection changes how much gradient reaches early layers -- it does not change how large a step a given lr takes once that gradient arrives.

6. A Short LR Sweep, Then the Full Rematch

ACTUAL OUTPUT

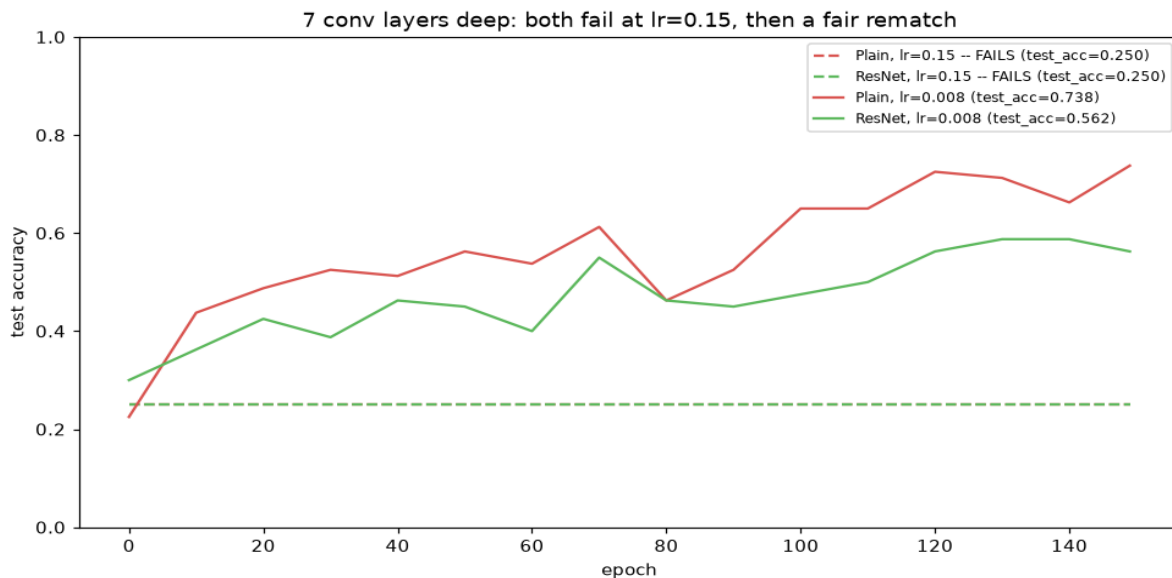
```
lr plain train_acc resnet train_acc
0.15 0.2500 0.2500
0.05 0.3550 0.4050
0.02 0.6000 0.5650
0.008 0.5700 0.6000
```

chosen lr for the full rematch: 0.008 (best 25-epoch training accuracy for ResNetStyleConvNet)

ACTUAL OUTPUT

```
network params train_acc test_acc
PlainDeepConvNet 7012 1.0000 0.7375
ResNetStyleConvNet 7012 1.0000 0.5625
```

At the recalibrated $lr=0.008$, both networks fully converge on the training set ($train_acc=1.000$) -- but PlainDeepConvNet actually generalizes BETTER ($test_acc=0.7375$) than ResNetStyleConvNet ($test_acc=0.5625$), despite the gradient-flow diagnostic's real, measured advantage for the residual version, and despite both networks sharing identical parameter counts. The lesson isn't "residual connections make any learning rate work" -- section 5 already showed that's false at this depth -- and it isn't "residual connections always win on accuracy" either. It's that a shortcut connection measurably changes how gradient is distributed across depth, which is real and worth understanding on its own terms, but a distinct effect from picking a better learning rate, and neither guarantees the more sophisticated architecture wins on a given dataset at a given scale -- the same caveat Days 47, 49, and 52 already taught with their own architectures.



Dashed lines: both networks flatlined at chance level at $lr=0.15$. Solid lines: the $lr=0.008$ rematch -- Plain (red) climbs above ResNet (green) by the end of training.

7. What This Maps To in PyTorch

```

class ResidualBlock(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)

    def forward(self, x):
        out = torch.relu(self.conv1(x))
        out = self.conv2(out) # no relu yet
        return torch.relu(out + x) # add the shortcut, THEN relu

```

Real ResNet also adds a batch-norm layer after each conv (Day 54's topic) and a learned 1x1-conv projection on the shortcut whenever a block changes channel count or downsamples -- neither is needed here since every block in this lesson keeps the channel width C fixed.

Interview Preparation -- Technical Questions

Q: Why does a residual connection help with vanishing gradients?

A: Because the block computes $\text{out} = \text{ReLU}(F(x) + x)$, the local derivative of the sum w.r.t. x is exactly 1 (in addition to whatever dF/dx contributes). Backpropagating through many stacked residual blocks means gradient always has this unattenuated additive path back to earlier layers, instead of being purely a product of each layer's (potentially small) Jacobian -- measured directly in this lesson as a stem-to-deepest-layer gradient ratio of 0.415 (residual) vs. 0.116 (plain), same initial weights.

Q: If residual connections help gradients flow, why did ResNetStyleConvNet still fail completely at $\text{lr}=0.15$?

A: Gradient flow and step size are two different things. A shortcut connection changes how MUCH gradient signal reaches an early layer; it does nothing to change how LARGE an update $\text{lr}=0.15$ applies once that gradient arrives. If the resulting step is still big enough to push a unit's pre-activation permanently negative, that unit dies under ReLU regardless of how the gradient that caused the step got there -- confirmed directly by measuring 100% dead fractions in both networks at that lr .

Q: Why must the two convs inside a residual block use padding?

A: The block computes $F(x) + x$, which requires $F(x)$ and x to have the exact same shape. A 3x3 "valid" (unpadded) convolution shrinks spatial size by 2 pixels per conv, so two stacked unpadded 3x3 convs would produce a 4-pixel-smaller output than the block's input -- padding=1 per conv (new in this lesson's `conv2d_forward_mc`) exactly cancels that shrinkage, keeping $F(x)$ and x shape-compatible.

Q: ResNetStyleConvNet has a real, measured gradient-flow advantage but LOSES on test accuracy to PlainDeepConvNet at the fair rematch lr . Is that a contradiction?

A: No -- they're different claims. The gradient-flow diagnostic is about optimization dynamics (how easily a network CAN be trained at all, especially at aggressive learning rates or greater depth). Final test accuracy also depends on generalization, which is dataset- and scale-dependent. Day 52 showed the exact same pattern with VGGStyle's confirmed parameter savings not translating into better accuracy -- a real, verified structural property of an architecture doesn't automatically win on every metric.

Interview Preparation -- Theory Questions

Q: What specific architectural change did He et al. (2015) introduce, and what problem was it aimed at?

A: Identity shortcut (skip) connections around blocks of convolutional layers, so a block learns a residual function $F(x) = H(x) - x$ instead of the full mapping $H(x)$ directly. It was aimed at enabling much deeper networks (50, 101, 152+ layers) to train successfully, where plain stacks of that depth had been empirically observed to perform WORSE than shallower ones, not just slower to train.

Q: This lesson found that residual connections don't automatically fix a too-aggressive learning rate. What does that imply about how ResNet's real success was achieved?

A: That solving depth in practice needed more than one idea working together -- ResNet's original architecture also pairs shortcuts with careful initialization and batch normalization (Day 54's topic) after every conv. This lesson isolates the shortcut connection alone specifically to show what it does and does not do by itself, rather than crediting it with the full effect.

Q: Why is measuring gradient norm at initialization a cleaner test of 'vanishing gradients' than measuring final test accuracy?

A: Final test accuracy is affected by many confounded factors (learning rate, epochs, dataset size, generalization). Measuring the gradient norm reaching each layer on a single forward+backward pass, with both networks starting from identical weights, isolates ONE variable -- how the architecture itself distributes gradient magnitude across depth -- before training-dynamics confounds enter the picture.

Key Takeaway and What's Next

Key takeaway: Residual (skip) connections give backpropagation an unattenuated additive gradient path through every block, a real and directly measured effect (stem-to-deepest-layer gradient ratio 0.415 vs. 0.116, identical initial weights) that costs zero extra parameters. But that effect is distinct from learning-rate calibration -- an aggressive enough lr can still kill both a plain and a residual network's units -- and it does not guarantee better test accuracy on every dataset at every scale: at this lesson's scale, the plain network actually generalized better once both were fairly calibrated.

What's next: Day 54 introduces batch normalization inside CNNs, extending Day 40's dense-layer batchnorm to conv layers -- another tool, alongside residual connections, that real deep networks combine to train reliably at depths far beyond what either idea achieves alone.

Suggested watch order: 1) ResNet Explained Simply, 2) Vanishing & Exploding Gradient explained.